LIN7076 – Foundations of Computational Linguistics

Feedforward Neural Networks

Adèle Hénot-Mortier

25/11/2025

Queen Mary University of London

Plan for today

- Introduce the XOR-problem through the lens of pragmatic inferences
- Extend the concept of logistic regression to feedforward neural nets
- Show how such nets can capture non-linear patterns, and XOR-related patterns in particular

The XOR problem

Literal interpretation of *or*

- Suppose Jo is taking a literature class, and to pass this class Jo must write **one** short essay about Anna Karenina or War and Peace.
- Jo may read only one book and write the essay it, or read both books, and write an essay about which book they prefer to write about.
- We are wondering if Jo will pass the class. Jo's classmate Al knows the book(s) Jo read and tells us:

(1) Jo will pass, because Jo read Anna Karenina or War and Peace.

- Would Al be considered a liar if Jo read only Anna Karenina? Only War and Peace? Both books?
- Intuition: here it does not matter that much if Jo read only one book or both books, because in both cases Jo will pass. So we tend to interpret *or* as compatible with *both*. This is the **literal, inclusive reading** of *or*.

Pragmatic interpretation of *or*

- Suppose Jo is taking a literature class. To get an A Jo must write **one** short essay about Anna Karenina or War and Peace, and to get an A+ Jo must write **two** essays, one about each book.
- We are wondering which grade Jo is aiming for. Jo's classmate Al knows the books Jo read and tells us:

(2) Jo read Anna Karenina or War and Peace.

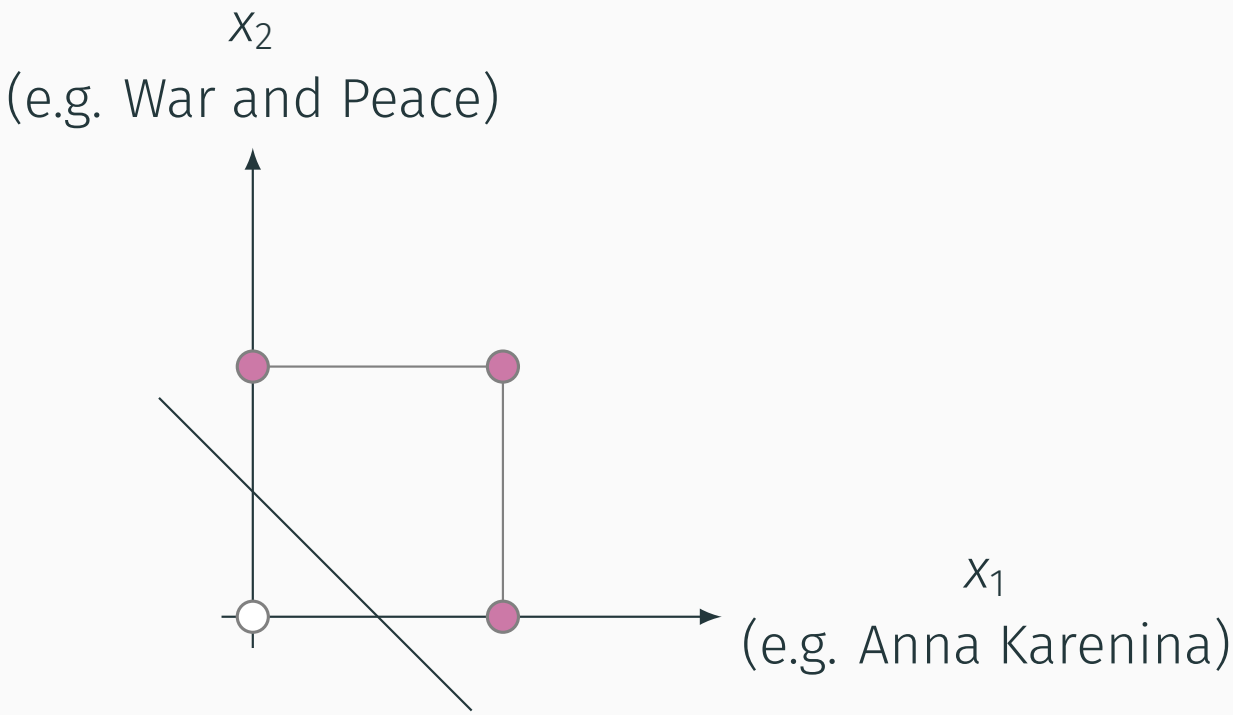
- Would Al be considered a liar if Jo read only Anna Karenina? Only War and Peace? Both books?
- Intuition: here it does matter if Jo read only one book or both books. So we tend to interpret *or* as incompatible with *both*, because if *both* were true, then it'd have been better to say so.
- This is the **pragmatic, exclusive reading** of *or*.

Summary of the literal and pragmatic interpretations

Anna Karenina	War and Peace	Literal *or*	Pragmatic *or*
0	0	0	0
0	1	1	1
1	0	1	1
1	1	**1**	**0**

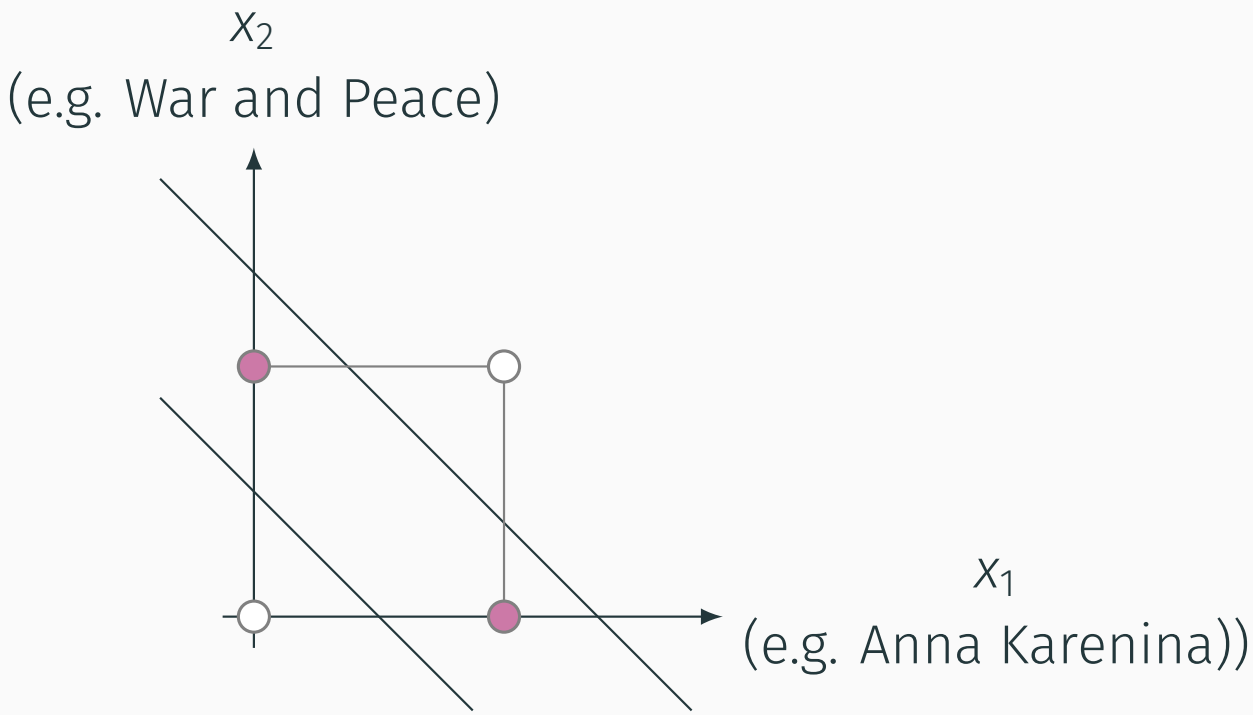
- Literal *or* is true whenever at least one disjunct is true. This is also called **inclusive or**.
- Pragmatic *or* is true if only one of the disjunct is true. This is also called **exclusive or**, or **XOR** for short.
- Which interpretation is the most likely depends on the context–whether the alternative *and*/*both* is made relevant.
  - If it's not, then using *or* does not signal *and* doe snot hold, and *or* is inclusive.
  - If it is relevant, then using *or* signals *and* does not hold, and *or* is exclusive.

Graphical representation of the inclusive reading



- Recall that 2D binary data is linearly separable (i.e. distinguishable *via* a logistic regression) if one can draw a line separating the positive and negative examples.
- Is the literal *or* data above linearly separable? Yes!

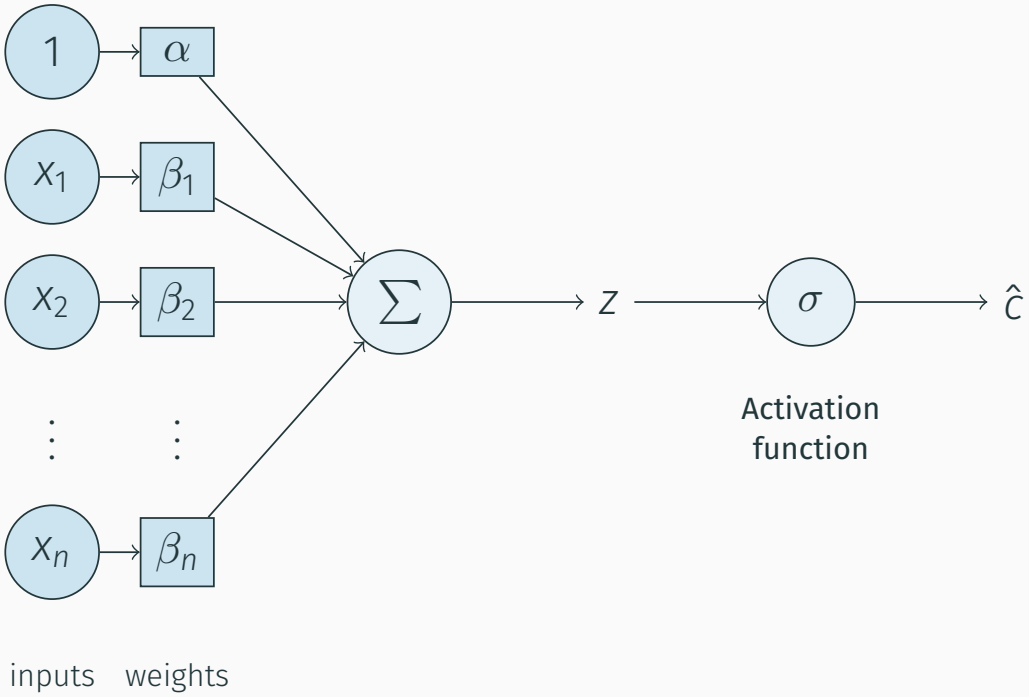
Graphical representation of the exclusive reading



- Is the pragmatic *or* data above linearly separable? No! The best we can do is separate one positive/negative cluster against the three others.
- We need two lines to fully separate the data.

Logistic regression reminder

- Logistic regression can be represented using the following net:



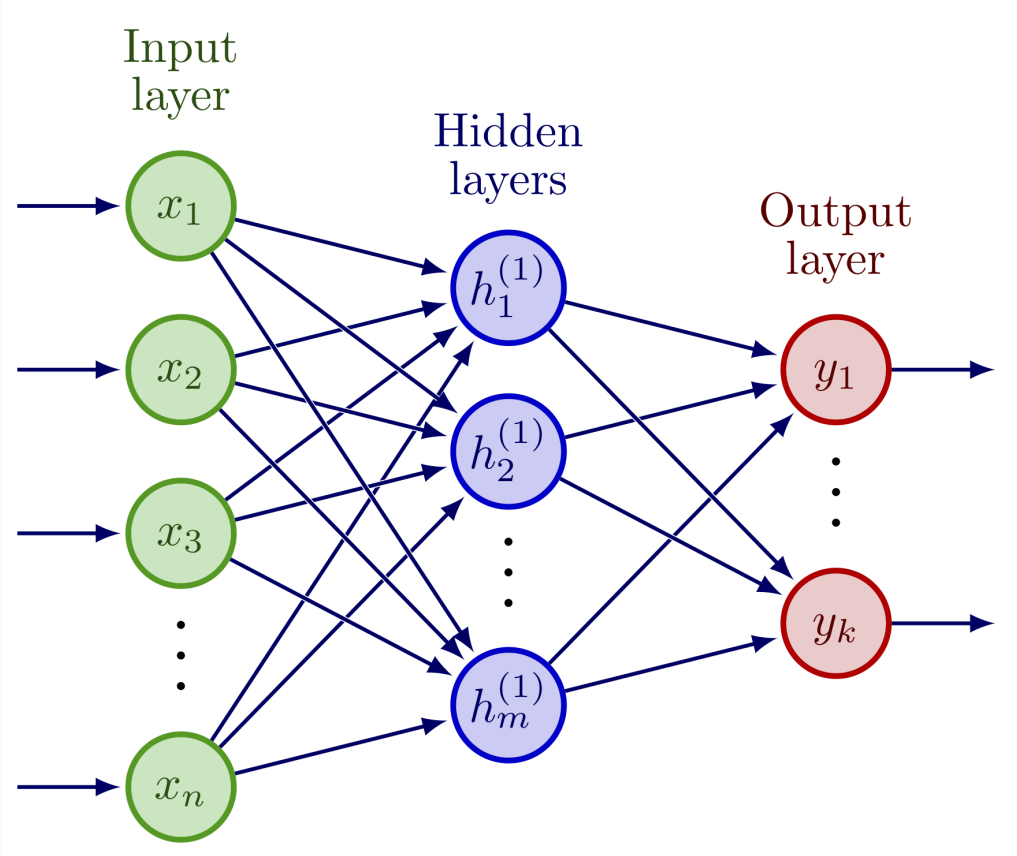
Logistic regression is a one-layer, one-unit neural net, with a sigmoid **activation function**.

- Let's generalize this to capture the pragmatic reading of *or*, XOR.

Feedforward neural nets

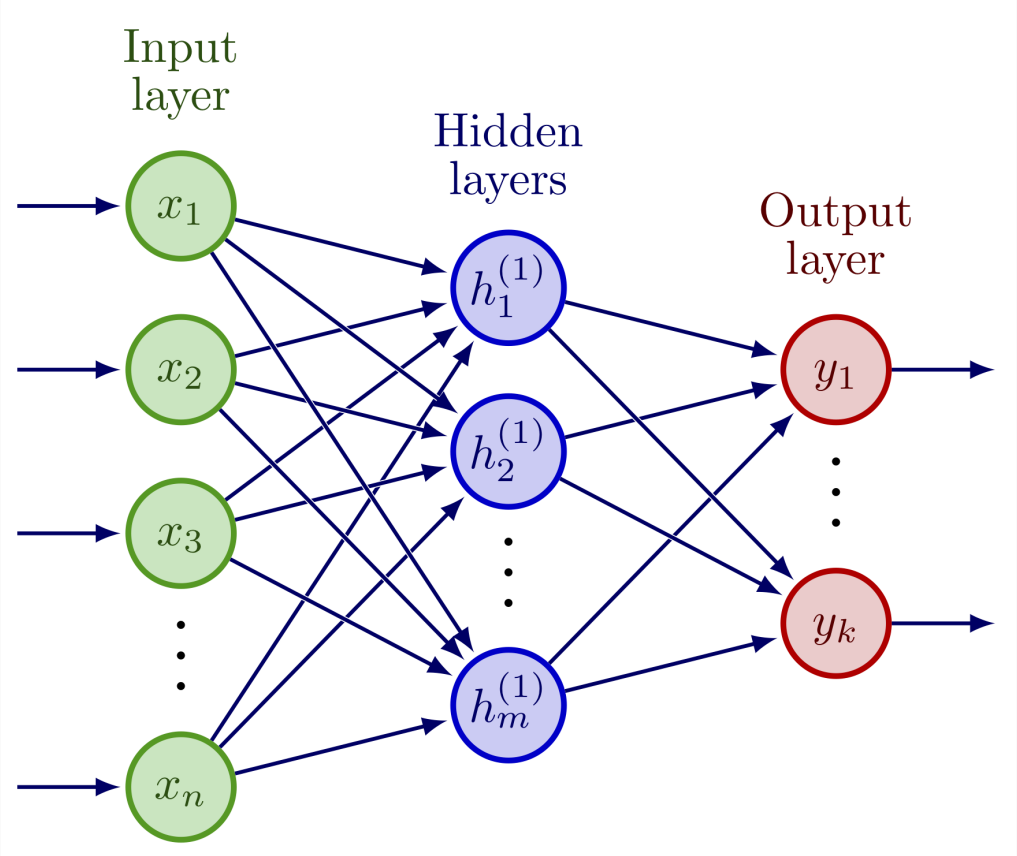
Feedforward neural nets as generalized regressions

- Roughly, basic "feed-forward" neural nets are stacked parallelized regressions, **except the non-linearity is not necessarily sigmoid**.



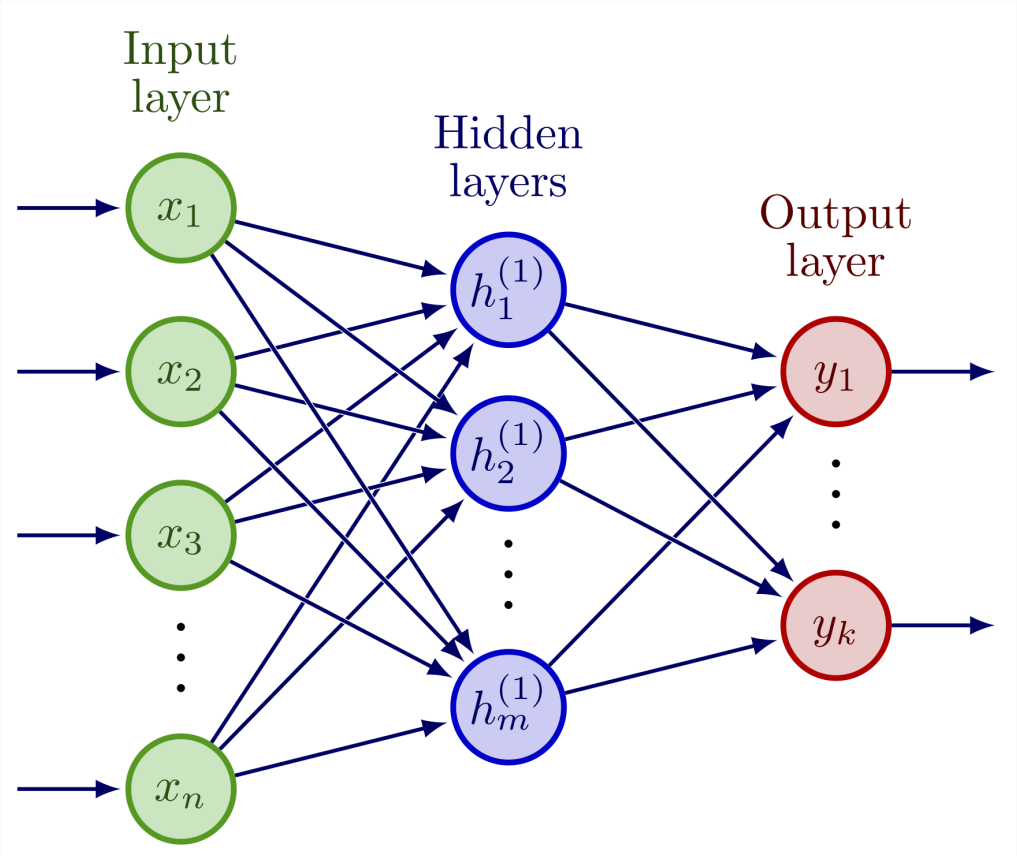
Feedforward neural nets as generalized regressions

- In the below plot, **arrows** represent **multiplications by a coefficient** (think β or α in the logistic regression model). Converging arrows sum.



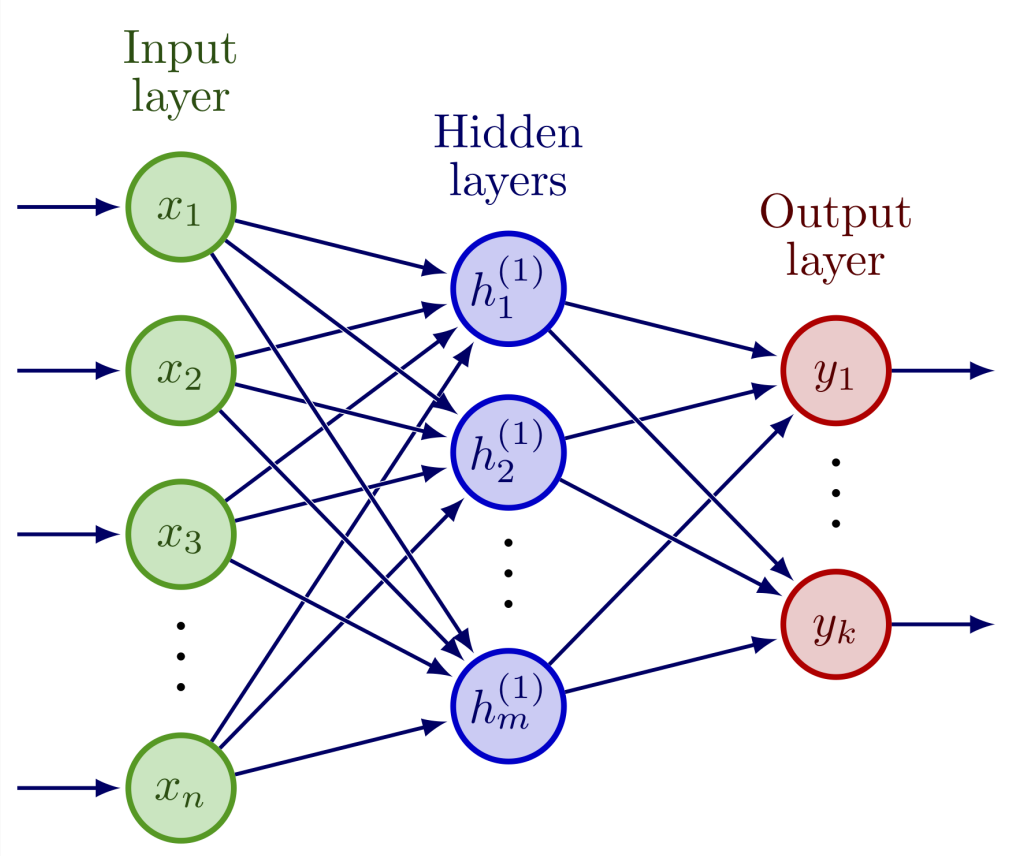
Feedforward neural nets as generalized regressions

- The circles are called **neurons** or **units**.
- They are arranged vertically to form **layers**.
- The below plot has one input layer, one hidden layer, and one output layer. There may be **more than one hidden layer**.



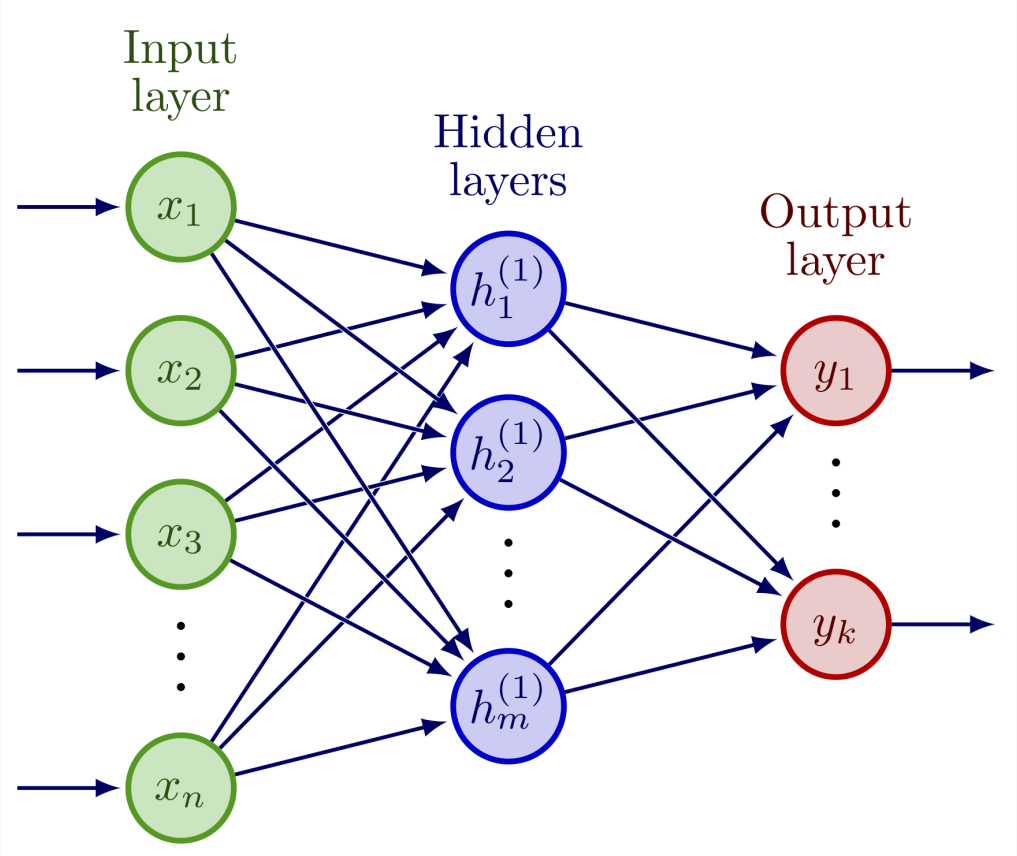
Feedforward neural nets as generalized regressions

- Hidden and output units are often followed by a **non-linear activation function** (e.g. **sigmoid**) that is implicit in the plot below.
- Different layers can be correspond to different activation functions.



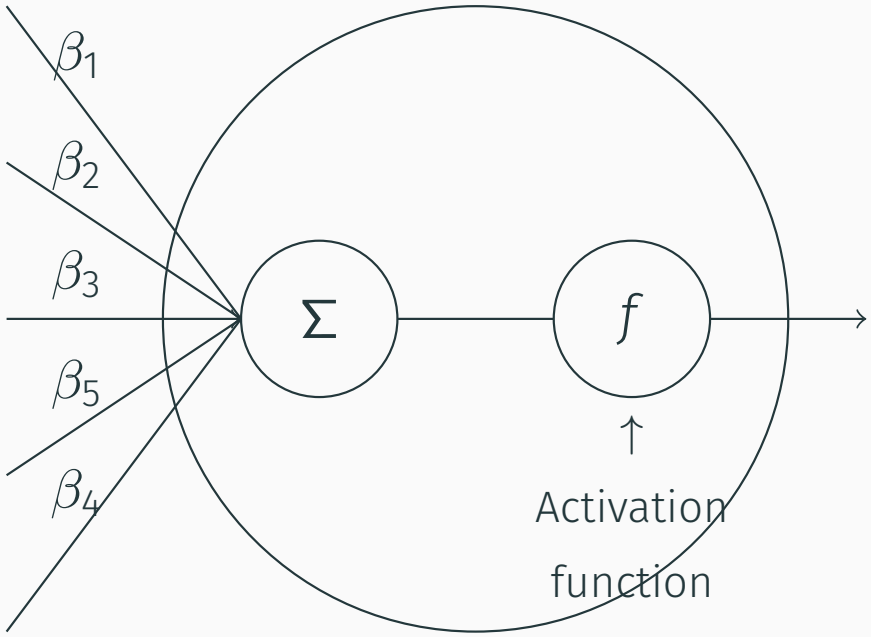
Feedforward neural nets as generalized regressions

- The neural net represented below is **feedforward**, because the arrows flow from left to right. There is **no retroaction/loop**.
- Next week we'll see neural nets with retroactions: RNNs and LSTMs.



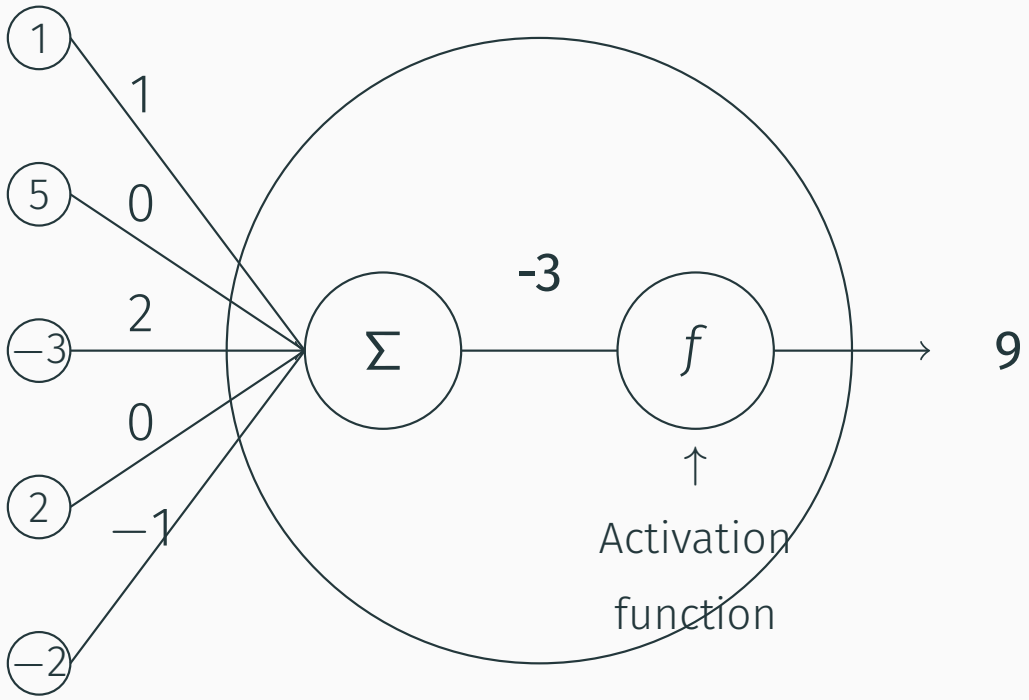
Activation functions

Inside a neuron



- Each neuron sums its weighted inputs (coming from preceding neurons) and applies an activation on top of this.

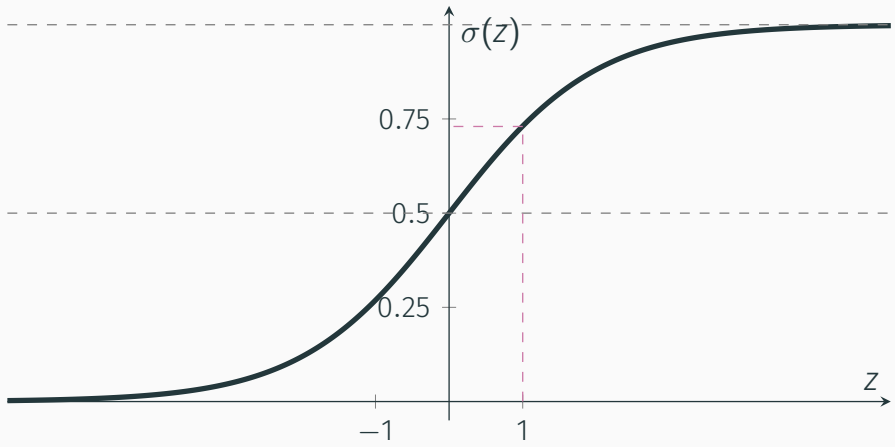
Neuron activation practice



- Suppose $f = x \mapsto x^2$. What does the main neuron output, based on the previous neurons (small circles) whose activations have been precomputed?
- First, we sum the weighted inputs:
  $1 \times 1 + 5 \times 0 + (-3) \times 2 + 2 \times 0 + (-2) \times (-1) = 1 - 6 + 2 = -3$.
- Then, we apply the activation: $f(-3) = (-3)^2 = 9$.

The sigmoid activation

- The **sigmoid** function ($\sigma(z) = \frac{1}{1+e^{-z}}$) is a "smooth" step function. It's equal to .5 for $z = 0$, converges to 0 for $z \to -\infty$ and to 1 for $z \to +\infty$.



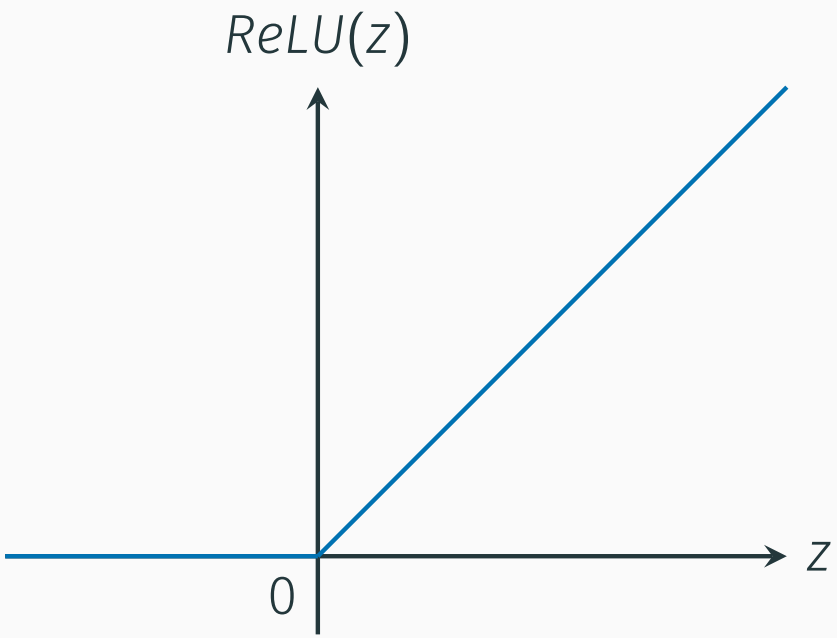
Thanks to ChatGPT

- $\sigma(z)$ essentially tells you **how probable class 1 is, for any z.** It's symmetric around point (0, .5), which means $\sigma(z) = 1 - \sigma(-z)$.[1]

[1] From an optimization perspective, it's also very nice because its slope has a nice formula equal to $\sigma(z) \times (1 - \sigma(z))$, for any z.

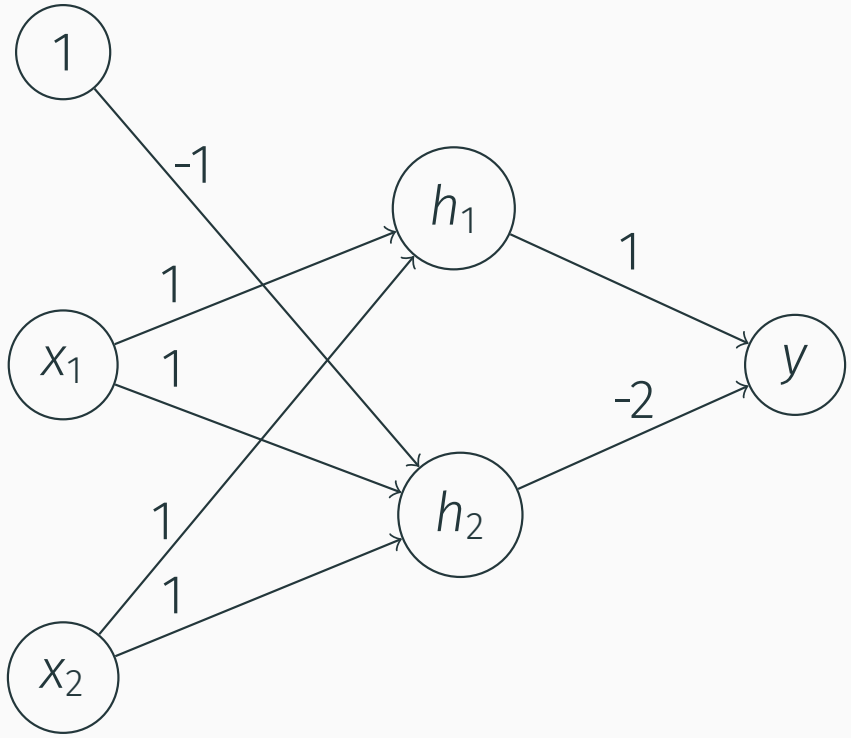
The ReLU

- The rectified linear unit (**ReLU** for short) is like a ramp: it's 0 if $z \leq 0$, and equal to z for $z \geq 0$. Can be summarized as $z \mapsto max(0, z)$.
- The ReLU's slope is defined almost everywhere. For $z = 0$, we can just set it to 1 or 0.



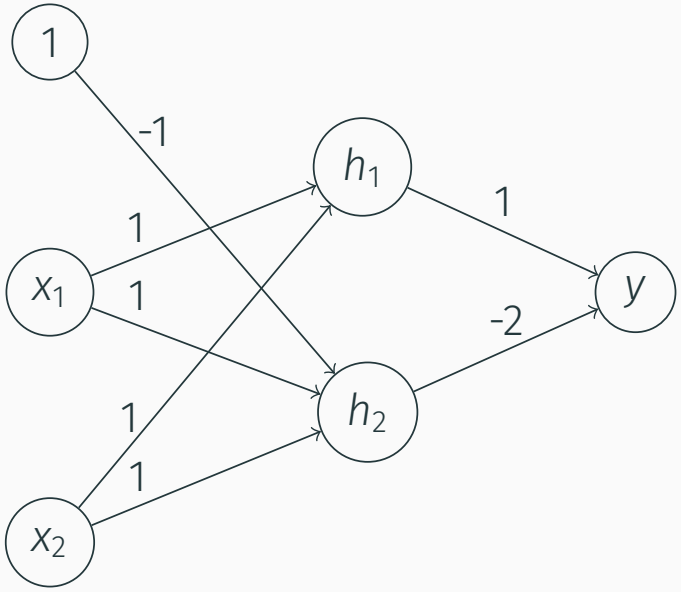
Solving XOR

Solving the XOR problem



- The input units x_1 and x_2 can be seen as A and B's truth values (binary variables) in a statement of the form *A or B*.
- We also have an input unit which is always 1 (think of it as the bias/intercept α in logistic regression).
- h_1, h_2 and y come with **ReLU activations**.

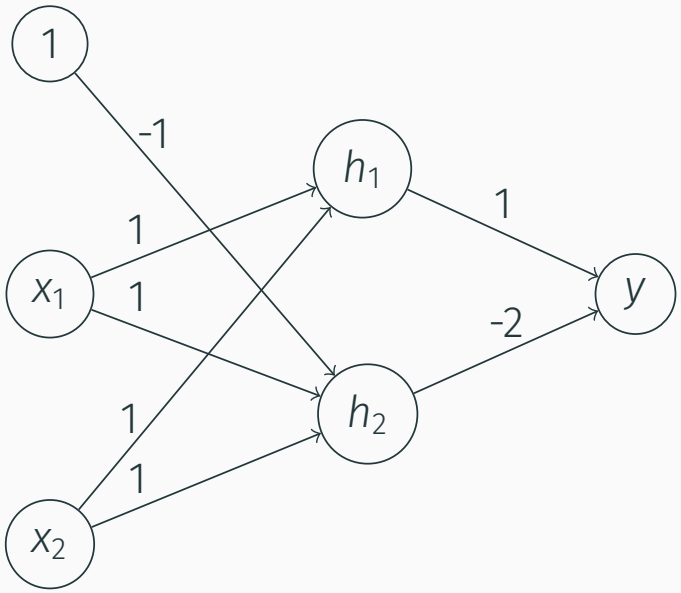
Solving the XOR problem: False-False case



$$ReLU(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{otherwise} \end{cases}$$

- If both x_1 and x_2 are False, then $x_1 = x_2 = 0$
- $h_1 = ReLU(1 \times 0 + 1 \times 0) = ReLU(0) = 0$
- $h_2 = ReLU(-1 \times 1 + 1 \times 0 + 1 \times 0) = ReLU(-1) = 0$
- $y = ReLU(1 \times h_1 + (-2) \times h_2) = ReLU(0 - 2 \times 0) = 0$
- We correctly predict False

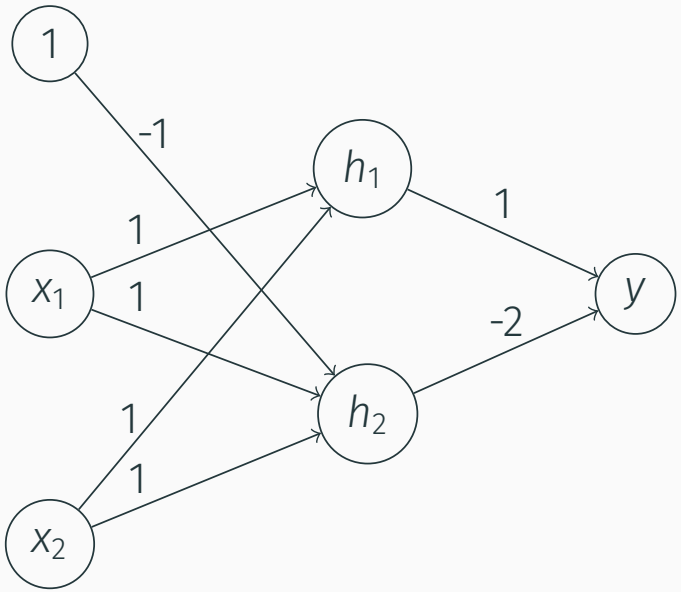
Solving the XOR problem: True-True case



$$ReLU(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{otherwise} \end{cases}$$

- If both x_1 and x_2 are True, then $x_1 = x_2 = 1$
- $h_1 = ReLU(1 \times 1 + 1 \times 1) = ReLU(2) = 2$
- $h_2 = ReLU(-1 \times 1 + 1 \times 1 + 1 \times 1) = ReLU(-1 + 2) = 1$
- $y = ReLU(1 \times h_1 + (-2) \times h_2) = ReLU(2 - 2 \times 1) = 0$
- We correctly predict False

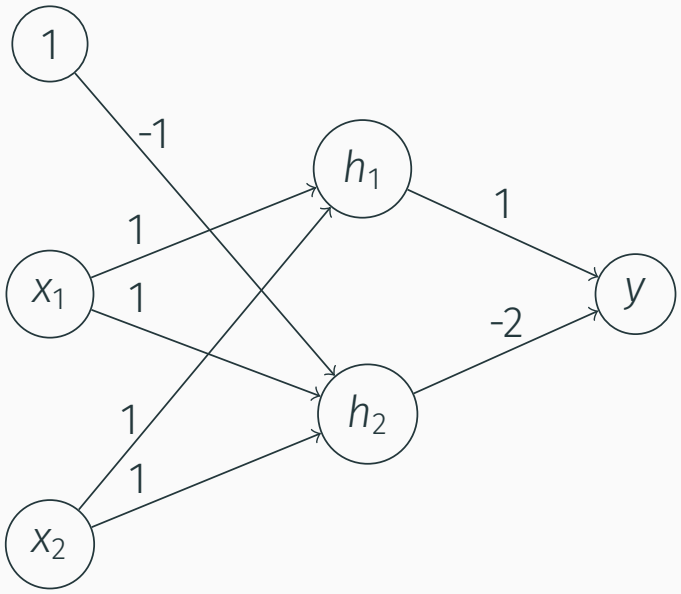
Solving the XOR problem: True-False case



$$ReLU(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{otherwise} \end{cases}$$

- If x_1 is True and x_2 is False, then $x_1 = 1$ and $x_2 = 0$
- $h_1 = ReLU(1 \times 1 + 1 \times 0) = ReLU(1) = 1$
- $h_2 = ReLU(-1 \times 1 + 1 \times 1 + 1 \times 0) = ReLU(0) = 0$
- $y = ReLU(1 \times h_1 + (-2) \times h_2) = ReLU(1 - 2 \times 0) = 1$
- We correctly predict True

Solving the XOR problem: False-True case



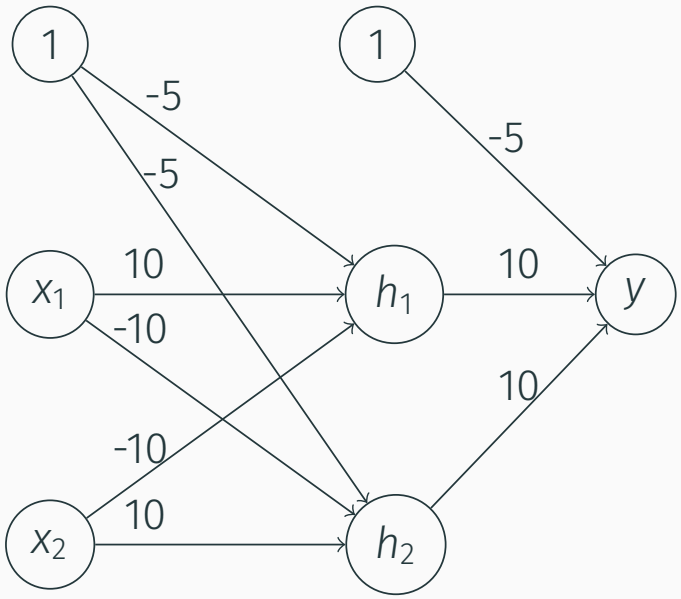
$$ReLU(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{otherwise} \end{cases}$$

- If x_1 is False and x_2 is True, then $x_1 = 0$ and $x_2 = 1$
- $h_1 = ReLU(1 \times 0 + 1 \times 1) = ReLU(1) = 1$
- $h_2 = ReLU(-1 \times 1 + 1 \times 0 + 1 \times 1) = ReLU(0) = 0$
- $y = ReLU(1 \times h_1 + (-2) \times h_2) = ReLU(1 - 2 \times 0) = 1$
- We correctly predict True

Summary

- By just adding **one hidden layer** to a standard regression model and **a few non-linearities**, we could capture the non-linear XOR pattern of the pragmatic interpretation of *or*.
- There are other neural net solutions to the XOR problem, some using a sigmoid activation at the end instead of a ReLU.
- What hidden layers do is successively transform the input in a non-linear way, to eventually end up in a warped space where the data is more separable.

A sigmoid-based solution



- The above net models XOR assuming the hidden units h_1 and h_2, as well as the output unit y, come with sigmoid activations.
- Intuitively, h_1 fires iff only x_1 does, and h_2 fires iff only x_2 does.
- Then, y fires if either h_1 or h_2 does.
- Decomposes x_1 XOR x_2 as $(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$, i.e, "only x_1 or only x_2".

Neural Nets and natural language

Universal Approximation Theorems

- Neural networks with a certain structure can, in principle, approximate any continuous function to any desired degree of accuracy: they can be **universal approximators**.
- In the case of neural net with a single hidden layer with a sigmoid or ReLU activation (like the XOR network we just saw), universality can be achieved by increasing the number of hidden neurons.
- One can also achieve universality by increasing the number of layers.
- These theorems are existence proofs and do not provide a full-proof method for finding the network's precise architecture and parameters that achieve the right degree of approximation.

Is Universal Approximation good news?

- At first blush, the fact that big-enough neural nets can mimic any function seems to suggests they are fit to model linguistic competence.
- Maybe humans learning a language are just very elaborate neural nets! And maybe that's why we can learn the pragmatic reading of *or*.
- Two very important caveats imo: **learnability**; **overgeneration**.

Learnability: neural nets

- As pointed out earlier, Universal Approximation Theorems do not provide a full-proof method for finding the network's precise architecture and parameters that achieve the right degree of approximation.
- In other words, we cannot predict how hard and costly it will be to learn a given pattern with a neural net; we just know it's possible.

Learnability: child language

- Natural language acquisition is fairly homogeneous across languages; even between spoken and signed variants. The same concepts tend to be hard/easy, and often, we can come up with good conceptual explanation as to why things are this way.
- Additionally, kids acquire language with fairly limited, noisy input, and do not appear to display the same computing capacities as neural nets!
- This difference in efficiency and transparency suggests we are maybe not Universal Approximators; instead, we come with a blueprint of what natural language should look like, guiding our learning.

Overgeneration: unattested paradigms and constructions

- More fundamentally, being able to approximate *anything* is *not* good news!
- There are many patterns that natural language *never*, or very rarely displays.

(3) Ban on hyperraising:
Jo seems [that is likely to win].

(4) *ABA in comparative and superlatives:

Good, **bett**er, ***good**est.	*ABA
Good, **bett**er, **b**est.	ABB
Fast, **fast**er, **fast**est	AAA

(5) No *wh*-extraction from inside complex subjects:
*Who did [a picture that belonged to] cause a scandal?

(6) No first-person imperatives …

- Big-enough neural nets are expected to learn those.

Overgeneration: humans often “regularize” languages

- When exposed to idiosyncratic or noisy languages, humans tend to regularize the input in systematic ways.
- Pidgins are contact languages that develops between groups that do not share a common language; they tend to be limited in terms of syntax and vocabulary.
- A pidgin becomes a creole once it becomes the native language of a community speakers.
- Creole languages are just as complex and expressive as English, implying that their first native speakers “chose” to not fully approximate the pidgin their learned from!
- A famous example is that of the Nicaraguan Sign Language.
- It’s a bit unclear if neural nets, even with limited data and architecture, would come up with the same regularization.

Lab

https://colab.research.google.com/drive/1WIMNdIHrebiRIwJOGe-hqig0GudfJaVXG?usp=sharing

Extras: optimizing neural nets

Optimizing logistic regression

- To optimize a logistic regression model, we needed a loss function telling us how well we are doing. This was the **cross-entropy loss**:

$$L_{\text{CE}} = -(\underbrace{c \times log(\sigma(z))}_{\neq 0 \text{ iff } c=1} + \underbrace{(1-c) \times log(1-\sigma(z))}_{\neq 0 \text{ iff } c=0})$$

- This function (which was a function of the model parameters α and βs) had to be **minimized**.
- We discussed how gradient descent could achieve this, by updating α and the βs in a direction decreasing the loss.
- **This can be generalized to neural nets, provided that the activation functions we choose are differentiable, i.e. have well-defined slopes everywhere.**

Backpropagation in neural nets

- Neural nets also come with a loss, which depends on the shape of the desired output. The loss tells us how much the predicted output differs from the gold-standard.
- This allows us to update the coefficients coming from the last hidden layer: just use gradient descent.
- The coefficients related to previous layers can also be updated: this is done by propagating the gradient of the final loss "backwards". This is called **backpropagation**.
- Backpropagation can be performed provided that the activation functions are differentiable, i.e. have a well-defined slope everywhere.